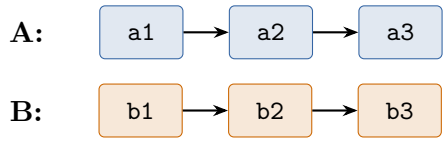
Guide: Recursive `merge_orders()`

TIP103 — Unit 3, Problem 4

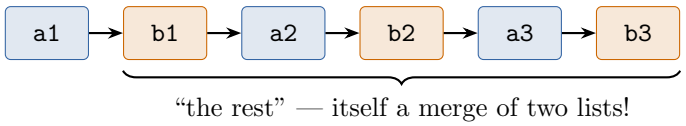
This guide walks you through *how to think* about the problem. It contains no solution code; you write the Python yourself.

Step 1: Understand the pattern with a picture

Draw both lists:



Goal:



The result starts with `a1`, and everything after `a1` is `b1` → `a2` → `b2` → `a3` → `b3`.

Ask yourself

That leftover part is a merge of two lists. **Which** two lists, and in **which order**? (Hint: whose node comes first in the leftover part?)

Step 2: Base cases (you already have these)

- If A is empty, the answer is just B.
- If B is empty, the answer is just A.

These stop the recursion.

Step 3: The "leap of faith"

Recursion works when you **trust that the function already works on a smaller input**. Assume `merge_orders(X, Y)` correctly merges any two *shorter* lists. Then ask:

1. What is the smaller problem? (From Step 1: something built from what comes *after* `a1`, plus list B.)
2. What order do the arguments go in, so that a `b` node comes next?
3. Is at least one argument **shorter** than before? If not, you will recurse forever — exactly the bug in your current line 28.

Step 4: Connect the pieces

Once the recursive call hands back "the rest of the merged sandwich":

Ask yourself

- Which node's `.next` should point to that result?
- What do you return as the head of the whole thing?

Watch out

If you overwrite a pointer before you have used what it pointed to, you lose the rest of that list. Think about the **order** of your steps. Do you need to save anything in a variable first?

Step 5: Trace it by hand

Before running, trace these on paper. For each call, write down the arguments and what gets returned.

A	B	Expected
`x`	`y`	`x → y`
`x → z`	`y`	`x → y → z`
`x`	`y → w`	`x → y → w`

If your traces match, run the real tests.

Step 6: Fix the test code

Your merge **changes the original nodes**. After the first test runs, `sandwich_a` is no longer `Bacon → Lettuce → Tomato`, so the second test will not print the expected output.

Ask yourself

How can you make sure each test starts with fresh lists?

Step 7: Complexity

Let n be the length of A and m be the length of B.

- **Time:** How much work happens in a single call, not counting the recursive call? How many calls happen in total?
- **Space:** You create no new nodes, but recursion uses the **call stack**. How many calls can be waiting on the stack at the same time?

Write both answers in terms of n and m, with a one-sentence reason for each.